



**SANTO[®]
TOMÁS**
INSTITUTO PROFESIONAL

Sistema de biblioteca

Asignatura: Patrones de Diseño

Integrantes: Eduardo Cona y Juan Pablo Saavedra

Docente: Osvaldo Archile

Fecha de elaboración: 19-06-2026

Índice

Descripción del Proyecto.....	3
1.1 Introducción.....	3
1.2 Objetivo general	3
1.3 Objetivos Específicos.....	3
1.4 Alcance del Sistema.....	4
1.5 Tecnologías Utilizadas	4
1.6 Arquitectura General	4
Requerimientos Funcionales.....	5
2.1 Gestión de Libros.....	5
2.2 Gestión de Autores	5
2.3 Gestión de Categorías	5
2.4 Gestión de Socios.....	5
2.5 Gestión de Préstamos	6
2.6 Notificaciones del Sistema.....	6
Casos de uso	7
Caso de Uso 1: Registrar Préstamo de Libro	7
Caso de Uso 2: Registrar Devolución de Libro	9
Caso de Uso 3: Actualización Reactiva de Disponibilidad (Observer)	10
Diagramas UML	11
Diagrama de Clases Estructural	11
Diagrama de Secuencia: Flujo de Préstamo Exitoso	12
Justificación de Patrones.....	13
Casos de Prueba (Pruebas Unitarias).....	14
Evidencias de Ejecución.....	16
Capturas de Pantalla.....	17
Conclusiones.....	21

Descripción del Proyecto

1.1 Introducción

El proyecto consiste en el desarrollo de un Sistema de biblioteca, que fue diseñado para facilitar la gestión de los recursos bibliográficos y administrativos. La aplicación permite la administración de libros, autores, categorías, socios y préstamos mediante una interfaz gráfica desarrollada en Java.

Este sistema fue desarrollado utilizando POO, una arquitectura basada en el Modelo-Vista-Controlador y patrones de diseño Gang of Four, con el propósito de mejorar la organización, mantenibilidad y escalabilidad del software.

1.2 Objetivo general

El objetivo general fue desarrollar un sistema de biblioteca funcional que permita administrar los procesos principales de una biblioteca mediante operaciones CRUD, utilizando patrones Gang of Four y las buenas prácticas de desarrollo de software

1.3 Objetivos Específicos

- Implementar una aplicación de escritorio
- Gestionar información de libros, autores, categorías y socios.
- Administrar préstamos y devoluciones de libros.
- Aplicar patrones de diseño Gang of Four para mejorar la estructura.
- Realizar pruebas unitarias que permitan validar el funcionamiento de los componentes críticos.
- Utilizar una base de datos relacional para el almacenamiento persistente de la información.

1.4 Alcance del Sistema

El sistema debe permitir registrar, modificar, eliminar y consultar información relacionada con libros, autores, categorías y socios. Además, incorpora funcionalidades para registrar préstamos y devoluciones, manteniendo una constancia en la actualización sobre la disponibilidad de los libros.

La aplicación está orientada a entornos de biblioteca de pequeña y mediana escala, permitiendo una administración centralizada de los recursos bibliográficos.

1.5 Tecnologías Utilizadas

Tecnología	Descripción
<i>Java</i>	Lenguaje de programación principal.
<i>Netbeans</i>	Entorno de desarrollo.
<i>MySQL</i>	Sistema gestor de base de datos.
<i>JDBC</i>	Conexión entre Java y MySQL.
<i>JUnit 5</i>	Pruebas unitarias
<i>Swing</i>	Desarrollo de la interfaz gráfica.

1.6 Arquitectura General

El sistema fue desarrollado siguiendo una arquitectura Modelo-Vista-Controlador, permitiendo separar adecuadamente las responsabilidades de cada componente.

El modelo está compuesto por las entidades de negocio y reglas del sistema. La vista corresponde a la interfaz gráfica desarrollada mediante Swing. El controlador es representado principalmente por la clase BibliotecaFacade, que coordina las operaciones del sistema. Finalmente, la persistencia de datos se realiza mediante clases DAO conectadas a una base de datos MySQL.

Requerimientos Funcionales

2.1 Gestión de Libros

Requerimiento	Descripción
<i>RF-01: Registrar Libro</i>	Permite ingresar un nuevo libro al sistema almacenando su información correspondiente.
<i>RF-02: Modificar Libro</i>	Permite actualizar los datos de un libro registrado.
<i>RF-03: Eliminar Libro</i>	Permite eliminar un libro del sistema cuando sea necesario.
<i>RF-04: Consultar Libros</i>	Permite visualizar y buscar los libros registrados.

2.2 Gestión de Autores

Requerimiento	Descripción
<i>RF-05: Registrar Autor</i>	Permite ingresar nuevos autores al sistema.
<i>RF-06: Modificar Autor</i>	Permite actualizar la información de un autor existente.
<i>RF-07: Eliminar Autor</i>	Permite eliminar autores registrados.
<i>RF-08: Consultar Autores</i>	Permite visualizar la información de los autores registrados.

2.3 Gestión de Categorías

Requerimiento	Descripción
<i>RF-09: Registrar Categoría</i>	Permite crear nuevas categorías para clasificar libros.
<i>RF-10: Modificar Categoría</i>	Permite actualizar información de categorías existentes.
<i>RF-11: Eliminar Categoría</i>	Permite eliminar categorías registradas.
<i>RF-12: Consultar Categorías</i>	Permite visualizar las categorías disponibles.

2.4 Gestión de Socios

Requerimiento	Descripción
<i>RF-13: Registrar Socio</i>	Permite registrar nuevos socios de la biblioteca.
<i>RF-14: Modificar Socio</i>	Permite actualizar información de socios registrados.
<i>RF-15: Eliminar Socio</i>	Permite eliminar socios del sistema.
<i>RF-16: Consultar Socios</i>	Permite visualizar los socios registrado

2.5 Gestión de Préstamos

Requerimiento	Descripción
<i>RF-17: Registrar Préstamo</i>	Permite asociar un libro disponible a un socio.
<i>RF-18: Registrar Devolución</i>	Permite registrar la devolución de un libro prestado.
<i>RF-19: Consultar Préstamos</i>	Permite visualizar el historial de préstamos registrados.
<i>RF-20: Actualizar Disponibilidad</i>	Permite modificar automáticamente la disponibilidad de un libro cuando se realiza un préstamo o devolución.

2.6 Notificaciones del Sistema

Requerimiento	Descripción
<i>RF-21: Notificación de Cambios</i>	Permite informar a los componentes observadores cuando ocurre una modificación relacionada con préstamos o devoluciones.

Casos de uso

Caso de Uso 1: Registrar Préstamo de Libro

- **Actor Principal:** Bibliotecario / Administrador.
- **Precondiciones:**
 1. El socio debe existir en el sistema y su estado debe ser "Activo".
 2. El libro para prestar debe estar registrado y su cantidad disponible (cantidadDisponible) debe ser mayor a cero.
- **Flujo Principal:**
 1. El bibliotecario accede a la pestaña "Registrar Préstamo" en el formulario de préstamos (FrmPrestamo).
 2. Selecciona al socio por su RUT y el libro por su título.
 3. Define la fecha del préstamo y la fecha prevista de devolución.
 4. Presiona el botón "Registrar Préstamo".
 5. La interfaz gráfica delega la petición a la fachada única de negocio (BibliotecaFacade).
 6. La fachada realiza las validaciones correspondientes: comprueba que el socio esté "Activo" y que el stock del libro sea mayor a cero.
 7. El sistema registra el nuevo préstamo en la base de datos a través de DAOPrestamo con estado inicial "Activo" y multa inicial de 0.0.
 8. El sistema descuenta de forma automática en 1 unidad la cantidad disponible del libro en la base de datos a través de DAOLibro.
 9. La fachada notifica el cambio a las ventanas registradas como observadores.
 10. El sistema despliega un mensaje confirmando el éxito del registro.

- **Flujos Alternativos:**
 - **Flujo Alternativo A (Libro sin ejemplares disponibles):**
 - En el paso 6, el sistema detecta que la disponibilidad del libro es 0.
 - El sistema interrumpe el flujo arrojando una excepción `IllegalStateException`.
 - Se muestra una alerta de error al usuario y no se guarda ningún registro en la base de datos.
 - **Flujo Alternativo B (Socio Inactivo):**
 - En el paso 6, el sistema detecta que el socio seleccionado tiene un estado distinto de "Activo".
 - Se cancela la operación y se muestra una ventana de advertencia indicando que el socio no está habilitado para realizar préstamos.
- **Postcondiciones:** El préstamo queda registrado en estado "Activo" en la base de datos y la disponibilidad física del libro en stock disminuye en 1 unidad.

Caso de Uso 2: Registrar Devolución de Libro

- **Actor Principal:** Bibliotecario / Administrador.
- **Precondiciones:** Debe existir un préstamo activo registrado en el sistema.
- **Flujo Principal:**
 1. El bibliotecario abre la sección de "Devoluciones" en el formulario de préstamos (FrmPrestamo).
 2. Selecciona de la lista de préstamos activos el registro correspondiente.
 3. Ingresa la fecha real de devolución del ejemplar.
 4. Presiona el botón "Registrar Devolución".
 5. La fachada del sistema llama al cálculo automático de multas basándose en la diferencia entre la fecha de devolución real y la prevista.
 6. Si la fecha real es posterior a la prevista, se aplica una multa de **\$500** por cada día de atraso.
 7. El sistema actualiza el préstamo en la base de datos a estado "Devuelto", registrando la fecha real y el monto de la multa calculada.
 8. El sistema incrementa en 1 la disponibilidad del libro en la base de datos llamando a DAOLibro.
 9. La fachada notifica el evento a los observadores reactivos.
 10. El sistema muestra un panel con el resumen de la devolución e indica el cobro de la multa en caso de aplicar.
- **Postcondiciones:** El préstamo pasa a estado "Devuelto" y el ejemplar del libro es reintegrado al stock disponible de la biblioteca (cantidadDisponible aumenta en 1).

Caso de Uso 3: Actualización Reactiva de Disponibilidad (Observer)

- **Actor Principal:** Socio / Bibliotecario.
- **Precondiciones:** La ventana interna de búsqueda de libros (FrmBusquedaLibro) debe estar abierta.
- **Flujo Principal:**
 1. El usuario abre la ventana interna de búsqueda de libros.
 2. El formulario se registra automáticamente como un PrestamoObserver ante la fachada única.
 3. En paralelo, ocurre una transacción de préstamo o devolución en otra ventana.
 4. La fachada dispara la notificación pertinente a todos los observadores registrados en su lista.
 5. La ventana de búsqueda recibe la notificación reactivamente en su método onPrestamoRegistrado o onPrestamoDevuelto.
 6. Vuelve a consultar la base de datos automáticamente y actualiza la tabla visual en tiempo real, sin requerir acción manual del usuario.
- **Postcondiciones:** El usuario visualiza la disponibilidad exacta de los ejemplares de manera sincronizada.

Diagramas UML

Diagrama de Clases Estructural

Muestra las clases críticas del sistema y sus relaciones bajo la implementación de los patrones creacionales, estructurales y de comportamiento de GoF.

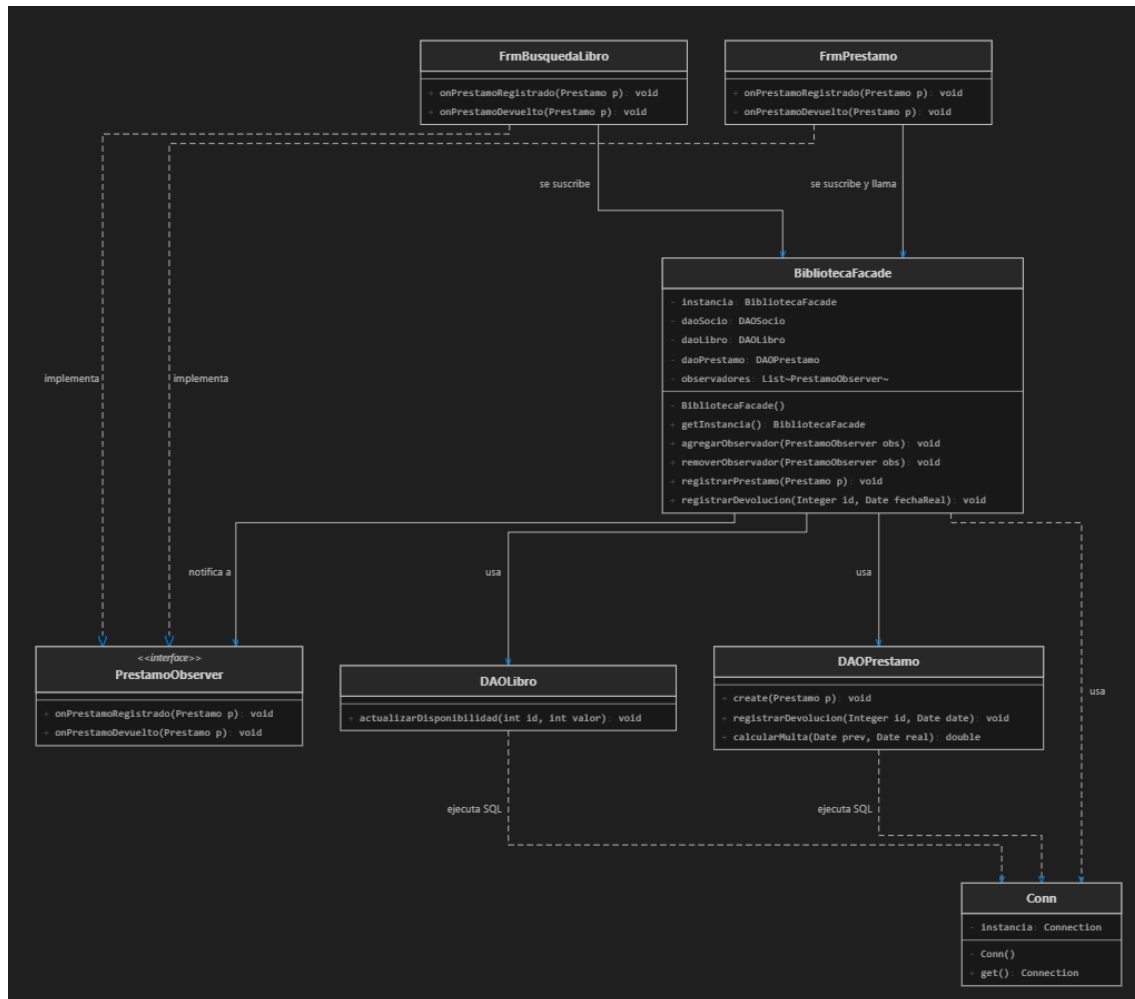
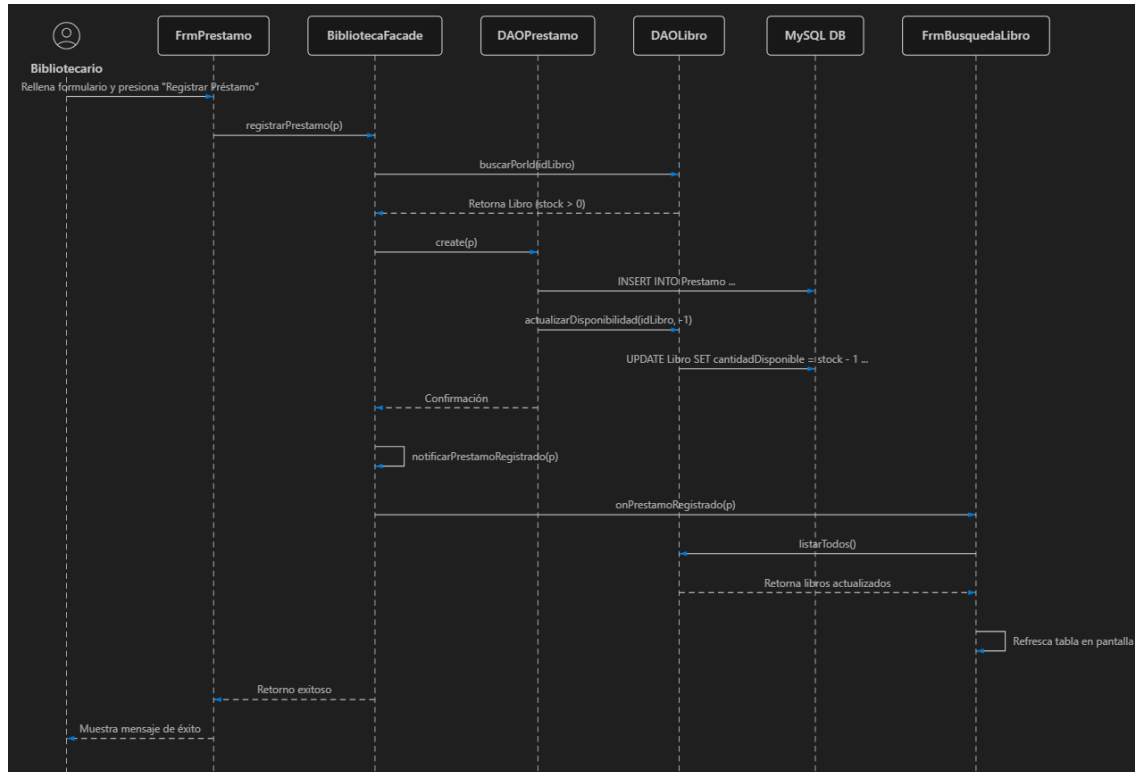


Diagrama de Secuencia: Flujo de Préstamo Exitoso

Representa las llamadas temporales desde que el bibliotecario opera en la GUI hasta la actualización física del stock del libro y la actualización reactiva en otras ventanas.



Justificación de Patrones

El uso de patrones de diseño GoF en el proyecto soluciona problemas críticos de acoplamiento, rendimiento y sincronización:

1. Singleton (Creacional - Conn.java)

- **Justificación:** Abrir y cerrar conexiones físicas a MySQL de forma redundante en cada acción de la interfaz degrada los tiempos de respuesta de la aplicación y puede saturar el servidor por exceso de hilos. El patrón Singleton garantiza que exista una única conexión física activa reutilizable en memoria para toda la sesión del usuario.

2. Facade (Estructural - BibliotecaFacade.java)

- **Justificación:** En ausencia de una Fachada, cada formulario Swing de la GUI tendría que instanciar múltiples DAOs para validar y realizar una sola operación (por ejemplo, validar el estado del socio en un DAO, validar el stock del libro en otro y registrar la transacción en un tercero). La Fachada desacopla por completo la capa de presentación de la persistencia de datos, ofreciendo a las ventanas de Swing una interfaz simplificada de negocio y centralizando todas las reglas críticas de validación.

3. Observer (Comportamiento - PrestamoObserver.java)

- **Justificación:** Los formularios de transacciones (FrmPrestamo) y los de consultas (FrmBusquedaLibro) son ventanas independientes que se ejecutan simultáneamente en el escritorio. El patrón Observer permite que un cambio de estado en la base de datos (como la disminución del stock de un libro al prestarlo) se notifique automáticamente a las ventanas interesadas para que refresquen su información en tiempo real de manera reactiva, eliminando acoplamientos rígidos entre formularios gráficos.

Casos de Prueba (Pruebas Unitarias)

La lógica del negocio y el acceso a los datos están validados mediante una suite de 10 casos de prueba automatizados con JUnit 5 en la clase BibliotecaFacadeTest.java.

ID Caso	Método del Test	Descripción de la Validación	Entrada de Datos	Resultado Esperado	Aserción Utilizada
CP-01	testSingletonConexionNotNull	Comprueba que la conexión de base de datos no sea nula y responda correctamente.	Petición a Conn.get().	Conexión JDBC activa y válida.	assertNotNull, assertTrue
CP-02	testSingletonConexionEsUnica	Valida que las llamadas a la conexión devuelvan la misma instancia física (Singleton).	Comparar dos referencias a Conn.get().	Ambas referencias son idénticas (conn1 == conn2).	assertSame
CP-03	testCalculaMultasSinRetraso	Comprueba que devolver un libro a tiempo no genere cargos de multa adicionales.	Fecha prevista: Hoy. Fecha real: Hoy.	Multa = \$0.0	assertEquals
CP-04	testCalculaMultasConRetraso	Valida que un retraso de 3 días aplique una multa de 1500(500 por día).	Fecha prevista: Hoy. Fecha real: Hoy + 3 días.	Multa = \$1500.0	assertEquals
CP-05	testBuscarSocioExistente	Busca en la BD al socio semilla por su RUT y valida sus datos básicos.	RUT: "12345678-9".	Nombre = "Juan Pérez González", Estado = "Activo".	assertNotNull, assertEquals
CP-06	testBuscarSocioInexistente	Valida el retorno de búsqueda cuando un socio no se encuentra en el sistema.	RUT: "0000000-0"	Objeto devuelto es null.	assertNull

CP-07	testSocioEstadoActivoPorDefecto	Valida que al insertar un nuevo socio sin estado definido, la fachada le asigne "Activo".	Socio de prueba con RUT "88888888-8".	Registro guardado con éxito en la BD con estado "Activo".	assertNotNull, assertEquals
CP-08	testBuscarAutorExistente	Comprueba la integridad del autor semilla en la BD.	ID Autor: 1.	Nombre del Autor = "Gabriel García Márquez".	assertNotNull, assertEquals
CP-09	testBuscarCategoriaExistente	Comprueba la integridad de la categoría semilla en la BD.	ID Categoría: 1.	Nombre de la Categoría = "Novela".	assertNotNull, assertEquals
CP-10	testRegistrarPrestamoSinStockLanzaExcepcion	Valida que el sistema interrumpa la operación y lance error al intentar prestar un libro sin stock.	Libro temporal con stock = 0; registrar préstamo.	Lanza excepción IllegalStateException y deniega registro.	

Evidencias de Ejecución

Preparación de la Base de Datos

- Importar y ejecutar el script `BSDSistemaBiblioteca.sql` en el motor de base de datos MySQL local para inicializar el esquema `sistema_biblioteca` junto con sus tablas relacionales e insertar los datos iniciales de prueba.
- Verificar que las credenciales en `Conn.java` (Usuario y Contraseña) correspondan a la configuración de su entorno MySQL local.

Comandos de Ejecución (Consola)

Ejecutar los siguientes comandos en la terminal desde el directorio raíz del proyecto:

1. **Compilar y construir el proyecto:**

```
bash  
mvn clean compile
```

2. **Ejecutar la suite de pruebas unitarias:**

```
bash  
mvn test
```

La consola reportará el paso exitoso de los 10 tests de JUnit 5 descritos en la sección de casos de prueba.

3. **Lanzar la interfaz gráfica:**

```
bash  
mvn exec:java
```

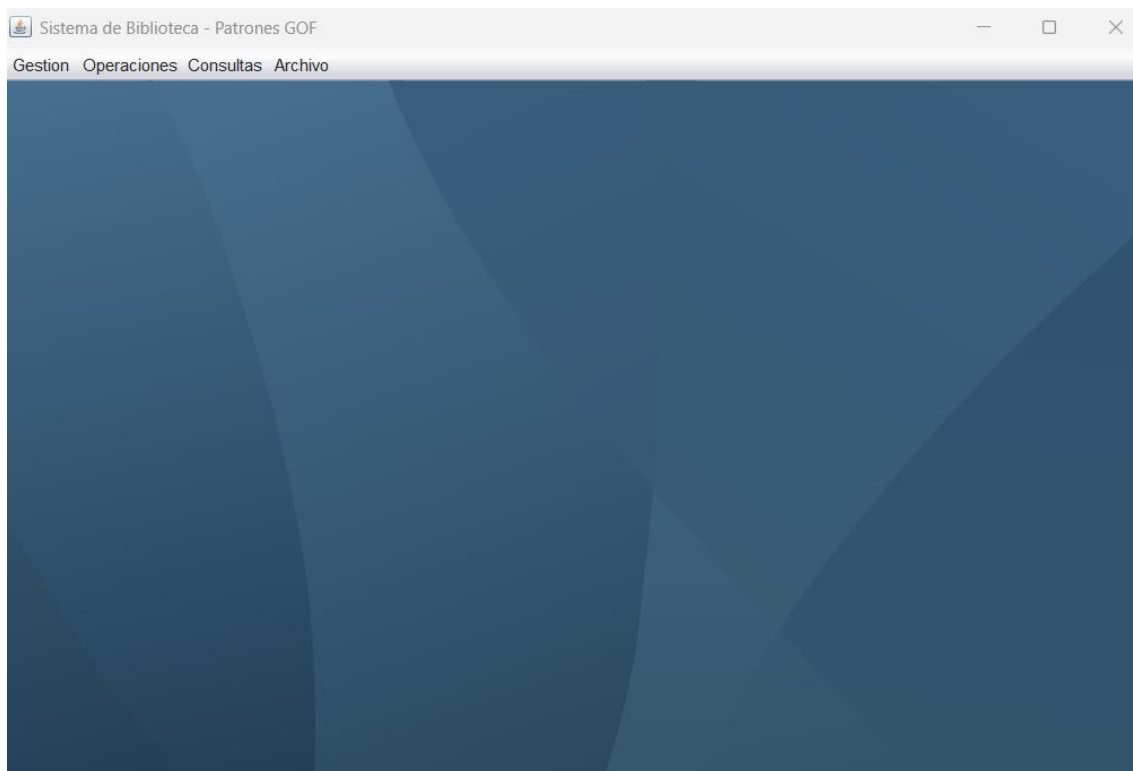
Comportamiento de las Ventanas Gráficas

- **JFrame Principal (Principal.java):** Actúa como contenedor de escritorio MDI (JDesktopPane) que centraliza y organiza los formularios internos.
- **FrmPrestamo:** Controla los flujos de préstamos (paneles dinámicos con comboboxes de socio y libro) y devoluciones (tabla de registros activos que calcula de manera instantánea las multas por días de atraso).
- **FrmBusquedaLibro:** Permite realizar consultas rápidas e incorpora la reactividad mediante `PrestamoObserver` para actualizar en tiempo real el stock remanente cuando ocurren transacciones en otras pestañas.

Capturas de Pantalla

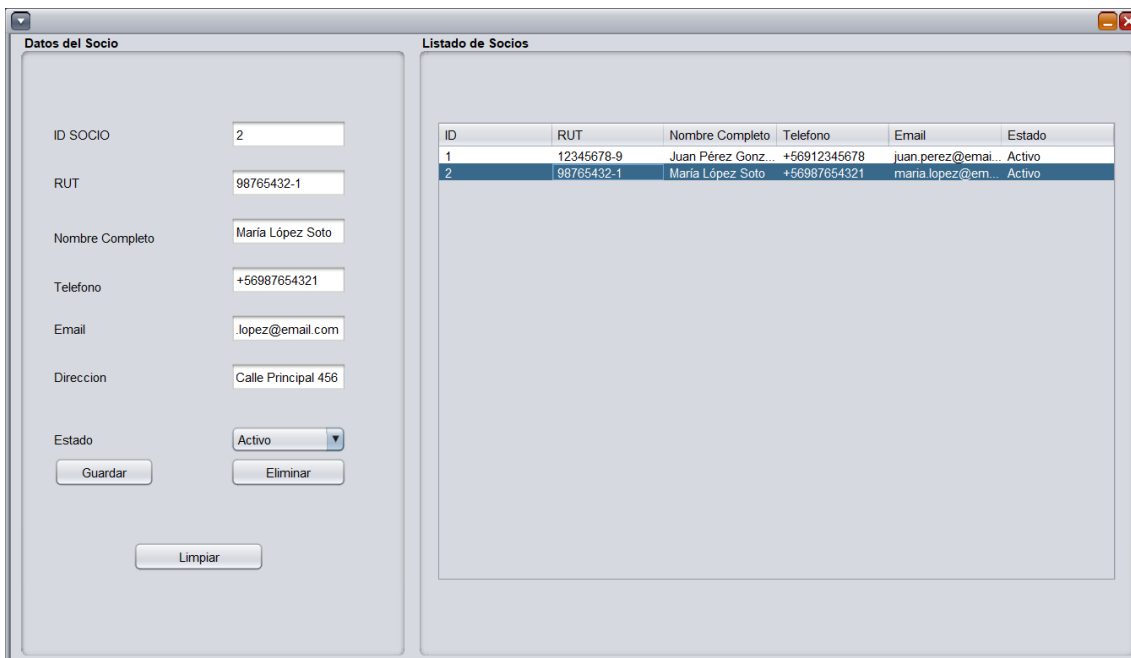
Captura 1: Ventana Principal

Interfaz gráfica MDI al ejecutar la aplicación por primera vez, mostrando la ventana del menú con las opciones de Gestión, Operaciones, Consultas y Archivos.



Captura 2: Formulario de Registro de Socio

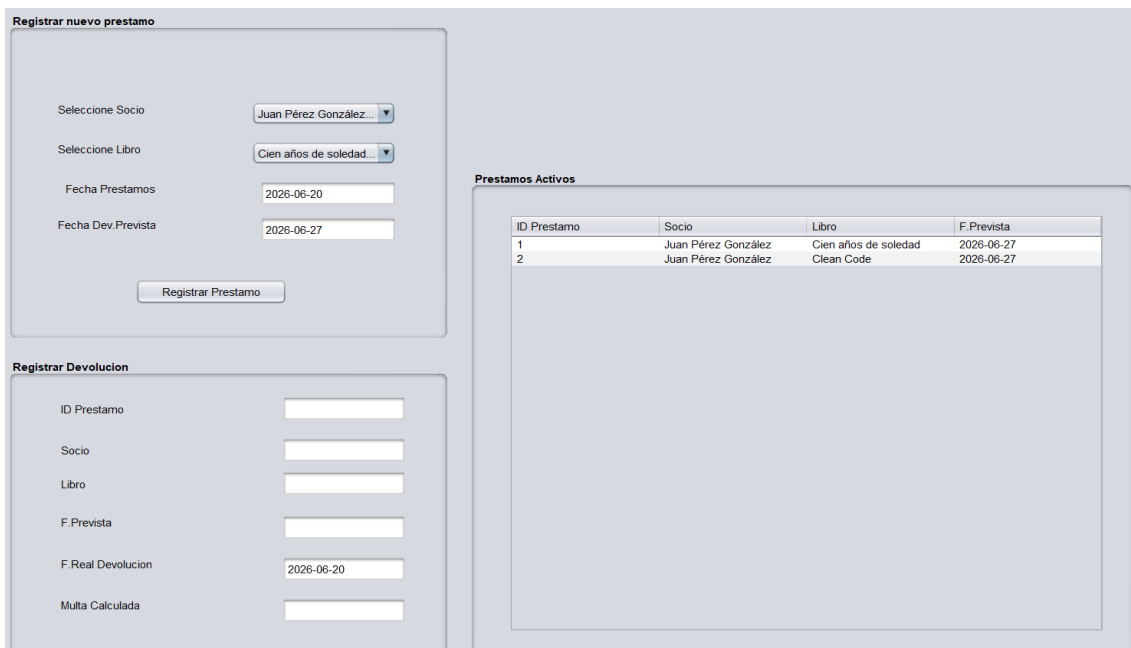
Vista de la ventana interna para registrar socios, mostrando los campos RUT, Nombre, Teléfono, Dirección, Email, el botón Guardar y la tabla inferior cargada con los socios registrados en la base de datos MySQL.



ID	RUT	Nombre Completo	Telefono	Email	Estado
1	12345678-9	Juan Pérez Gonz...	+56912345678	juan.perez@emai...	Activo
2	98765432-1	Maria López Soto	+56987654321	maria.lopez@em...	Activo

Captura 3: Registro de Préstamo Exitoso

Captura del panel "Registrar Préstamo", con los comboboxes de socio y libro seleccionados, la fecha de devolución programada y la ventana emergente de confirmación exitosa del préstamo (medio que no se ve por el tamaño de la PC).



ID Prestamo	Socio	Libro	F. Prevista
1	Juan Pérez González	Cien años de soledad	2026-06-27
2	Juan Pérez González	Clean Code	2026-06-27

Captura 4: Registro de Devolución y Cálculo de Multa

Vista de la pestaña "Devolución", seleccionando un préstamo vencido y mostrando la ventana emergente que detalla el cálculo del cargo de multa aplicado (ej. \$1500 por 3 días de retraso) al procesar la devolución.

Registrar Devolucion

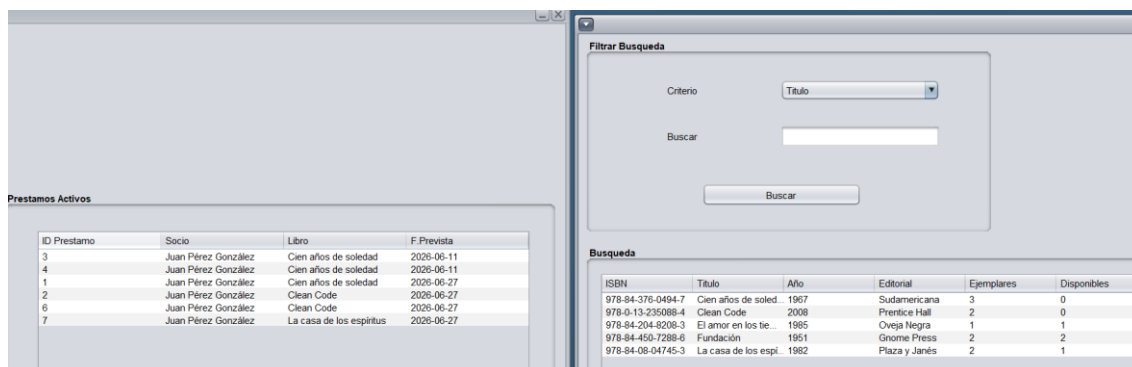
ID Prestamo	5
Socio	María López Soto
Libro	los tiempos del cólera
F.Prevista	2026-06-11
F.Real Devolucion	2026-06-20
Multa Calculada	\$4500.0

Información X

 Devolución completada.
Multa generada: \$4500.0

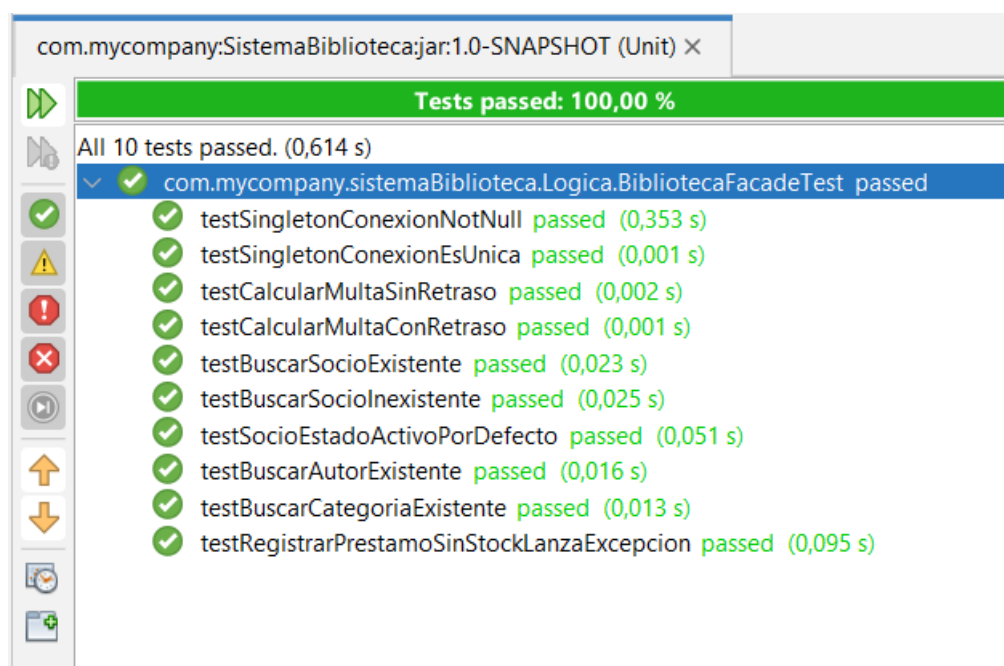
Captura 5: Sincronización Automática (Patrón Observer)

Dos ventanas internas abiertas en paralelo (FrmPrestamo y FrmBusquedaLibro). Se debe capturar el momento justo en que se presiona "Registrar Préstamo" y la cantidad disponible en la ventana de búsqueda disminuye reactivamente en tiempo real sin recargar manualmente.



Captura 6: Ejecución Exitosa de Pruebas Unitarias (JUnit 5)

Captura de la consola del sistema o el entorno IDE mostrando el resultado de la ejecución de mvn test con las 10 pruebas unitarias aprobadas exitosamente (Tests run: 10, Failures: 0, Errors: 0, Skipped: 0).



Conclusiones

1. **Impacto de los Patrones GOF en la Mantenibilidad:** La implementación del patrón **Facade** disminuyó drásticamente el acoplamiento entre la interfaz visual Swing y el motor de persistencia. Esto permite que cualquier cambio técnico en las bases de datos o en los DAOs no afecte la presentación de la aplicación, incrementando significativamente la escalabilidad y facilidad de mantenimiento.
2. **Interactividad Optimizada con Observer:** El patrón de comportamiento **Observer** proporciona una solución elegante e independiente para sincronizar componentes gráficos en tiempo real. Esto elimina la necesidad de implementar soluciones ineficientes de consulta constante (polling) o forzar al usuario a refrescar de forma manual la información.
3. **Rendimiento e Integridad de Recursos:** La combinación del patrón **Singleton** en la clase de conexión y la Fachada asegura el control óptimo de conexiones a MySQL. Se reduce la sobrecarga física del servidor garantizando la consistencia y exclusividad de la lógica en operaciones delicadas como el control de inventario y el cálculo de multas en la biblioteca.
4. **Respaldo mediante JUnit 5:** La suite de pruebas unitarias automatizadas proporciona una base sólida para el aseguramiento de la calidad del software. Garantiza la integridad de las reglas críticas del negocio en todo momento, facilitando la adición de nuevas funcionalidades en el futuro sin peligro de regresión.